

This model performs significantly better than the sequence-to-sequence model on the convex hull problem, but it is not applicable to problems where the output dictionary size depends on the input. Nevertheless, a very simple extension (or rather reduction) of the model allows us to do this easily.

2.3 Ptr-Net

We now describe a very simple modification of the attention model that allows us to apply the method to solve combinatorial optimization problems where the output dictionary size depends on the number of elements in the input sequence.

The sequence-to-sequence model of Section 2.1 uses a softmax distribution over a fixed sized output dictionary to compute $p(C_i|C_1, \dots, C_{i-1}, \mathcal{P})$ in Equation 1. Thus it cannot be used for our problems where the size of the output dictionary is equal to the length of the input sequence. To solve this problem we model $p(C_i|C_1, \dots, C_{i-1}, \mathcal{P})$ using the attention mechanism of Equation 3 as follows:

$$\begin{aligned} u_j^i &= v^T \tanh(W_1 e_j + W_2 d_i) \quad j \in (1, \dots, n) \\ p(C_i|C_1, \dots, C_{i-1}, \mathcal{P}) &= \text{softmax}(u^i) \end{aligned}$$

where softmax normalizes the vector u^i (of length n) to be an output distribution over the dictionary of inputs, and v , W_1 , and W_2 are learnable parameters of the output model. Here, we do not blend the encoder state e_j to propagate extra information to the decoder, but instead, use u_j^i as pointers to the input elements. In a similar way, to condition on C_{i-1} as in Equation 1, we simply copy the corresponding $P_{C_{i-1}}$ as the input. Both our method and the attention model can be seen as an application of content-based attention mechanisms proposed in [6, 5, 2].

We also note that our approach specifically targets problems whose outputs are discrete and correspond to positions in the input. Such problems may be addressed artificially – for example we could learn to output the coordinates of the target point directly using an RNN. However, at inference, this solution does not respect the constraint that the outputs map back to the inputs exactly. Without the constraints, the predictions are bound to become blurry over longer sequences as shown in sequence-to-sequence models for videos [12].

3 Motivation and Datasets Structure

In the following sections, we review each of the three problems we considered, as well as our data generation protocol.¹

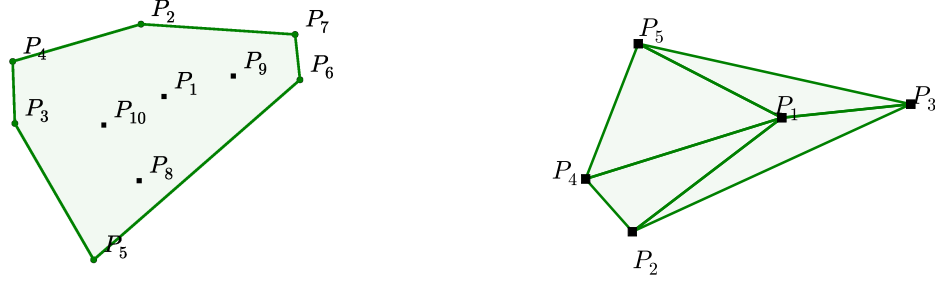
In the training data, the inputs are planar point sets $\mathcal{P} = \{P_1, \dots, P_n\}$ with n elements each, where $P_j = (x_j, y_j)$ are the cartesian coordinates of the points over which we find the convex hull, the Delaunay triangulation or the solution to the corresponding Travelling Salesman Problem. In all cases, we sample from a uniform distribution in $[0, 1] \times [0, 1]$. The outputs $\mathcal{C}^{\mathcal{P}} = \{C_1, \dots, C_{m(\mathcal{P})}\}$ are sequences representing the solution associated to the point set $\mathcal{P}$. In Figure 2, we find an illustration of an input/output pair $(\mathcal{P}, \mathcal{C}^{\mathcal{P}})$ for the convex hull and the Delaunay problems.

3.1 Convex Hull

We used this example as a baseline to develop our models and to understand the difficulty of solving combinatorial problems with data driven approaches. Finding the convex hull of a finite number of points is a well understood task in computational geometry, and there are several exact solutions available (see [13, 14, 15]). In general, finding the (generally unique) solution has complexity $O(n \log n)$, where n is the number of points considered.

The vectors $\mathcal{P}_j$ are uniformly sampled from $[0, 1] \times [0, 1]$. The elements C_i are indices between 1 and n corresponding to positions in the sequence $\mathcal{P}$, or special tokens representing beginning or end of sequence. See Figure 2 (a) for an illustration. To represent the output as a sequence, we start from the point with the lowest index, and go counter-clockwise – this is an arbitrary choice but helps reducing ambiguities during training.

¹We release all the datasets at <http://goo.gl/NDcOIG>.



(a) Input $\mathcal{P} = \{P_1, \dots, P_{10}\}$, and the output sequence $\mathcal{C}^{\mathcal{P}} = \{\Rightarrow, 2, 4, 3, 5, 6, 7, 2, \Leftarrow\}$ representing its convex hull. (b) Input $\mathcal{P} = \{P_1, \dots, P_5\}$, and the output $\mathcal{C}^{\mathcal{P}} = \{\Rightarrow, (1, 2, 4), (1, 4, 5), (1, 3, 5), (1, 2, 3), \Leftarrow\}$ representing its Delaunay Triangulation.

Figure 2: Input/output representation for (a) convex hull and (b) Delaunay triangulation. The tokens $\Rightarrow$ and $\Leftarrow$ represent beginning and end of sequence, respectively.

3.2 Delaunay Triangulation

A Delaunay triangulation for a set $\mathcal{P}$ of points in a plane is a triangulation such that each circumcircle of every triangle is empty, that is, there is no point from $\mathcal{P}$ in its interior. Exact $O(n \log n)$ solutions are available [16], where n is the number of points in $\mathcal{P}$.

In this example, the outputs $\mathcal{C}^{\mathcal{P}} = \{C_1, \dots, C_{m(\mathcal{P})}\}$ are the corresponding sequences representing the triangulation of the point set $\mathcal{P}$. Each C_i is a triple of integers from 1 to n corresponding to the position of triangle vertices in $\mathcal{P}$ or the beginning/end of sequence tokens. See Figure 2 (b).

We note that any permutation of the sequence $\mathcal{C}^{\mathcal{P}}$ represents the same triangulation for $\mathcal{P}$, additionally each triangle representation C_i of three integers can also be permuted. Without loss of generality, and similarly to what we did for convex hulls at training time, we order the triangles C_i by their incenter coordinates (lexicographic order) and choose the increasing triangle representation². Without ordering, the models learned were not as good, and finding a better ordering that the Ptr-Net could better exploit is part of future work.

3.3 Travelling Salesman Problem (TSP)

TSP arises in many areas of theoretical computer science and is an important algorithm used for microchip design or DNA sequencing. In our work we focused on the planar symmetric TSP: given a list of cities, we wish to find the shortest possible route that visits each city exactly once and returns to the starting point. Additionally, we assume the distance between two cities is the same in each opposite direction. This is an NP-hard problem which allows us to test the capabilities and limitations of our model.

The input/output pairs $(\mathcal{P}, \mathcal{C}^{\mathcal{P}})$ have a similar format as in the Convex Hull problem described in Section 3.1. $\mathcal{P}$ will be the cartesian coordinates representing the cities, which are chosen randomly in the $[0, 1] \times [0, 1]$ square. $\mathcal{C}^{\mathcal{P}} = \{C_1, \dots, C_n\}$ will be a permutation of integers from 1 to n representing the optimal path (or tour). For consistency, in the training dataset, we always start in the first city without loss of generality.

To generate exact data, we implemented the Held-Karp algorithm [17] which finds the optimal solution in $O(2^n n^2)$ (we used it up to $n = 20$). For larger n , producing exact solutions is extremely costly, therefore we also considered algorithms that produce approximated solutions: A1 [18] and A2 [19], which are both $O(n^2)$, and A3 [20] which implements the $O(n^3)$ Christofides algorithm. The latter algorithm is guaranteed to find a solution within a factor of 1.5 from the optimal length. Table 2 shows how they performed in our test sets.

²We choose $C_i = (1, 2, 4)$ instead of $(2, 4, 1)$ or any other permutation.

4 Empirical Results

4.1 Architecture and Hyperparameters

No extensive architecture or hyperparameter search of the Ptr-Net was done in the work presented here, and we used virtually the same architecture throughout all the experiments and datasets. Even though there are likely some gains to be obtained by tuning the model, we felt that having the same model hyperparameters operate on all the problems would make the main message of the paper stronger.

As a result, all our models used a single layer LSTM with either 256 or 512 hidden units, trained with stochastic gradient descent with a learning rate of 1.0, batch size of 128, random uniform weight initialization from -0.08 to 0.08, and L2 gradient clipping of 2.0. We generated 1M training example pairs, and we did observe overfitting in some cases where the task was simpler (i.e., for small n).

4.2 Convex Hull

We used the convex hull as the guiding task which allowed us to understand the deficiencies of standard models such as the sequence-to-sequence approach, and also setting up our expectations on what a purely data driven model would be able to achieve with respect to an exact solution.

We reported two metrics: accuracy, and area covered of the true convex hull (note that any simple polygon will have full intersection with the true convex hull). To compute the accuracy, we considered two output sequences $\mathcal{C}^1$ and $\mathcal{C}^2$ to be the same if they represent the same polygon. For simplicity, we only computed the area coverage for the test examples in which the output represents a simple polygon (i.e., without self-intersections). If an algorithm fails to produce a simple polygon in more than 1% of the cases, we simply reported FAIL.

The results are presented in Table 1. We note that the area coverage achieved with the Ptr-Net is close to 100%. Looking at examples of mistakes, we see that most problems come from points that are aligned (see Figure 3 (d) for a mistake for $n = 500$) – this is a common source of errors in most algorithms to solve the convex hull.

It was seen that the order in which the inputs are presented to the encoder during inference affects its performance. When the points on the true convex hull are seen “late” in the input sequence, the accuracy is lower. This is possibly the network does not have enough processing steps to “update” the convex hull it computed until the latest points were seen. In order to overcome this problem, we used the attention mechanism described in Section 2.2, which allows the decoder to look at the whole input at any time. This modification boosted the model performance significantly. We inspected what attention was focusing on, and we observed that it was “pointing” at the correct answer on the input side. This inspired us to create the Ptr-Net model described in Section 2.3.

More than outperforming both the LSTM and the LSTM with attention, our model has the key advantage of being inherently variable length. The bottom half of Table 1 shows that, when training our model on a variety of lengths ranging from 5 to 50 (uniformly sampled, as we found other forms of curriculum learning to not be effective), a single model is able to perform quite well on all lengths it has been trained on (but some degradation for $n = 50$ can be observed w.r.t. the model trained only on length 50 instances). More impressive is the fact that the model does extrapolate to lengths that it has never seen during training. Even for $n = 500$, our results are satisfactory and indirectly indicate that the model has learned more than a simple lookup. Neither LSTM or LSTM with attention can be used for any given $n' \neq n$ without training a new model on n' .

4.3 Delaunay Triangulation

The Delaunay Triangulation test case is connected to our first problem of finding the convex hull. In fact, the Delaunay Triangulation for a given set of points triangulates the convex hull of these points.

We reported two metrics: accuracy and triangle coverage in percentage (the percentage of triangles the model predicted correctly). Note that, in this case, for an input point set $\mathcal{P}$, the output sequence $\mathcal{C}(\mathcal{P})$ is, in fact, a set. As a consequence, any permutation of its elements will represent the same triangulation.